Complete Kubernetes Observability, Monitoring, Alerting, Backup & Restore Guide

Production-ready guide for a React + Fastify + Microservices + Amazon RDS PostgreSQL application running on Kubernetes.

1. Architecture Overview

Business Components:
- React Frontend
- Fastify API Gateway
- Authentication Service
- Church Member Service
- Events Service
- Notification Service
- Amazon RDS PostgreSQL

Observability Components:
- Filebeat (DaemonSet)
- Node Exporter (DaemonSet)
- kube-state-metrics
- Prometheus
- Grafana
- Elasticsearch
- Kibana
- Alertmanager
- Slack Notifications

2. Logging Architecture

Flow:
Fastify -> stdout -> Filebeat DaemonSet -> Elasticsearch -> Kibana

Implementation Steps:
1. Use structured JSON logging in Fastify with Pino.
2. Write logs to stdout.
3. Deploy Filebeat as a DaemonSet.
4. Configure Filebeat to collect container logs.
5. Send logs to Elasticsearch.
6. Create Kibana index patterns and dashboards.

3. Metrics Architecture

Metrics Sources:
- Node Exporter: Node CPU, memory, disk, filesystem, network.
- cAdvisor: Container CPU, memory, restart metrics.
- kube-state-metrics: Deployment, Pod, StatefulSet health.
- Fastify Metrics: Requests, latency, errors.

Flow:
Metrics Sources -> Prometheus -> Grafana

4. Fastify Application Metrics

Expose a /metrics endpoint using prom-client.
Track:
- http_requests_total
- http_request_duration_seconds

- http_errors_total
- active_requests

Create Grafana dashboards for:
- Requests per second
- API latency
- Error percentage
- Most frequently used endpoints

5. DaemonSets

Use DaemonSets for:
- Node Exporter
- Filebeat

Reason:
One instance must run on every node.

6. Alerting Strategy

Prometheus evaluates rules.
Alertmanager routes alerts to Slack.

Recommended Alerts:
- CPU > 80% for 5 minutes
- Memory > 85%
- Pod CrashLoopBackOff
- Pod restart count > 3
- Application unavailable
- High API latency
- High 5xx error rate
- Disk utilization > 80%

7. Grafana Dashboards

Infrastructure Dashboard:
- CPU
- Memory
- Network
- Disk

Kubernetes Dashboard:
- Pods
- Deployments
- StatefulSets
- Restarts

Application Dashboard:
- Request rate
- Error rate
- Response time
- Active users

8. Amazon RDS PostgreSQL Strategy

Why RDS:
- Managed backups
- Automated patching
- Monitoring
- High availability options
- Reduced operational burden

9. Backup Strategy

Layer 1: Automated Backups
- Enable automated backups.
- Retention 7-30 days.
- Enable Point-in-Time Recovery.

Layer 2: Manual Snapshots
Create snapshots before:
- Major deployments
- Schema changes
- Version upgrades

Layer 3: Logical Backups
Use pg_dump.
Store backups in Amazon S3.
Schedule daily or weekly exports.

10. Restore Procedures

Scenario A: Accidental data deletion
- Restore using Point-in-Time Recovery.

Scenario B: Corrupted database
- Restore latest snapshot.

Scenario C: Migration to a different environment
- Restore pg_dump backup from S3.

Test restores periodically.
Document Recovery Time Objective (RTO) and Recovery Point Objective (RPO).

11. Security Best Practices

- Enable TLS.
- Use RBAC.
- Store secrets in Kubernetes Secrets or AWS Secrets Manager.
- Restrict database access.
- Encrypt backup storage.
- Use least privilege IAM policies.

12. Deployment Sequence

Step 1: Deploy RDS PostgreSQL.
Step 2: Deploy Kubernetes cluster.
Step 3: Deploy microservices.
Step 4: Deploy Prometheus stack.
Step 5: Deploy Elasticsearch and Kibana.
Step 6: Deploy Filebeat DaemonSet.
Step 7: Configure Grafana dashboards.
Step 8: Configure Alertmanager and Slack.
Step 9: Enable backups.
Step 10: Conduct failure and restore testing.

13. Portfolio Talking Points

This project demonstrates:
- Kubernetes Administration
- AWS Architecture
- Infrastructure as Code
- Linux Operations
- Microservices
- Monitoring and Observability
- Logging and Alerting
- Database Reliability
- Backup and Disaster Recovery Planning
- Production Readiness

Reference Architecture